Gesture Following

1. Course Content

- Run example programs to automatically detect hand gestures in the image. The car will lock onto the detected gesture position (index finger root joint) and maintain a 0.4-meter distance for following movement, keeping it in the center of the image. When a "0" (fist) gesture is detected, tracking stops; other gestures continue tracking.

2. Running Examples

2.1 Startup Commands

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

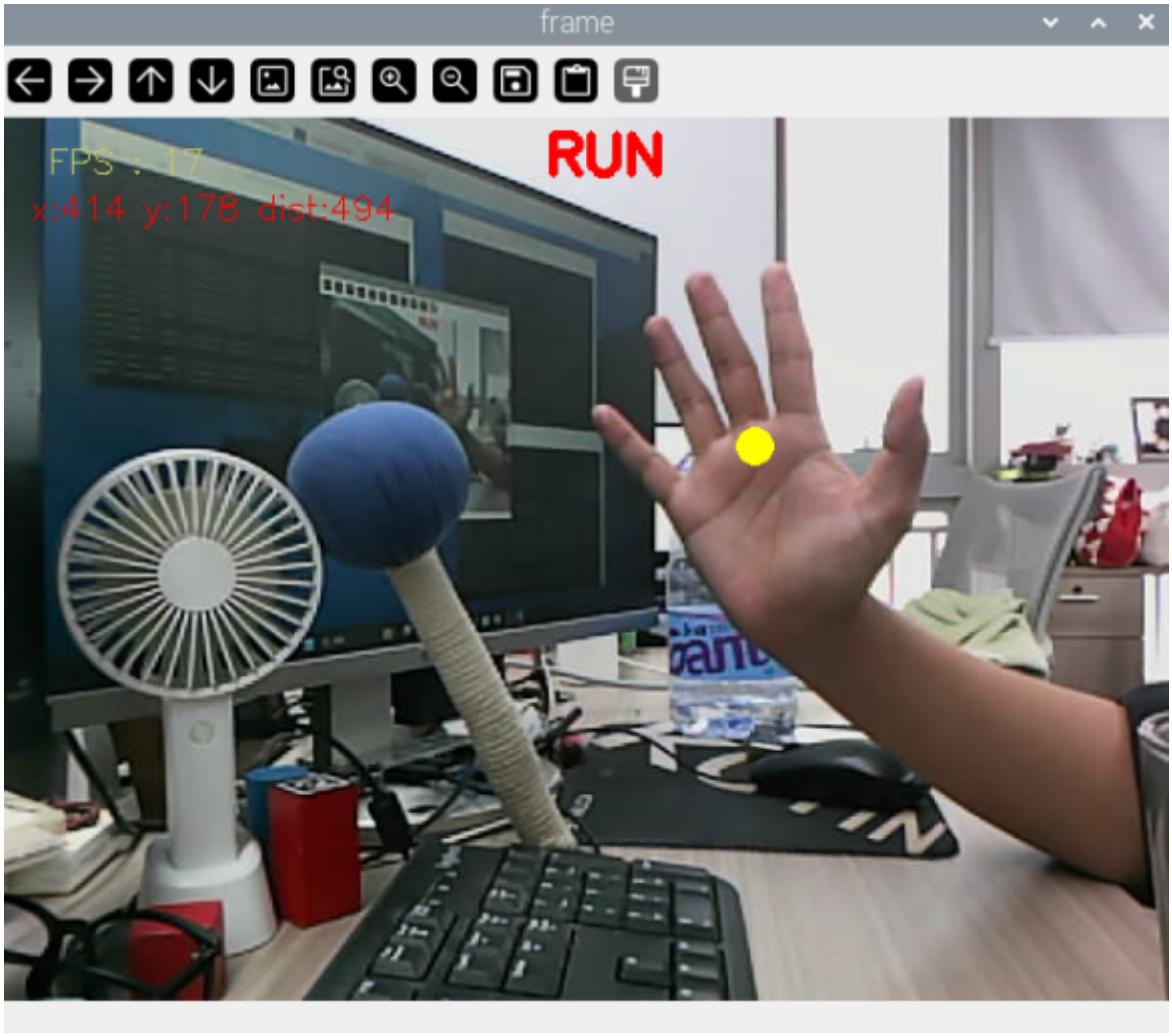
```
exec_m3
```

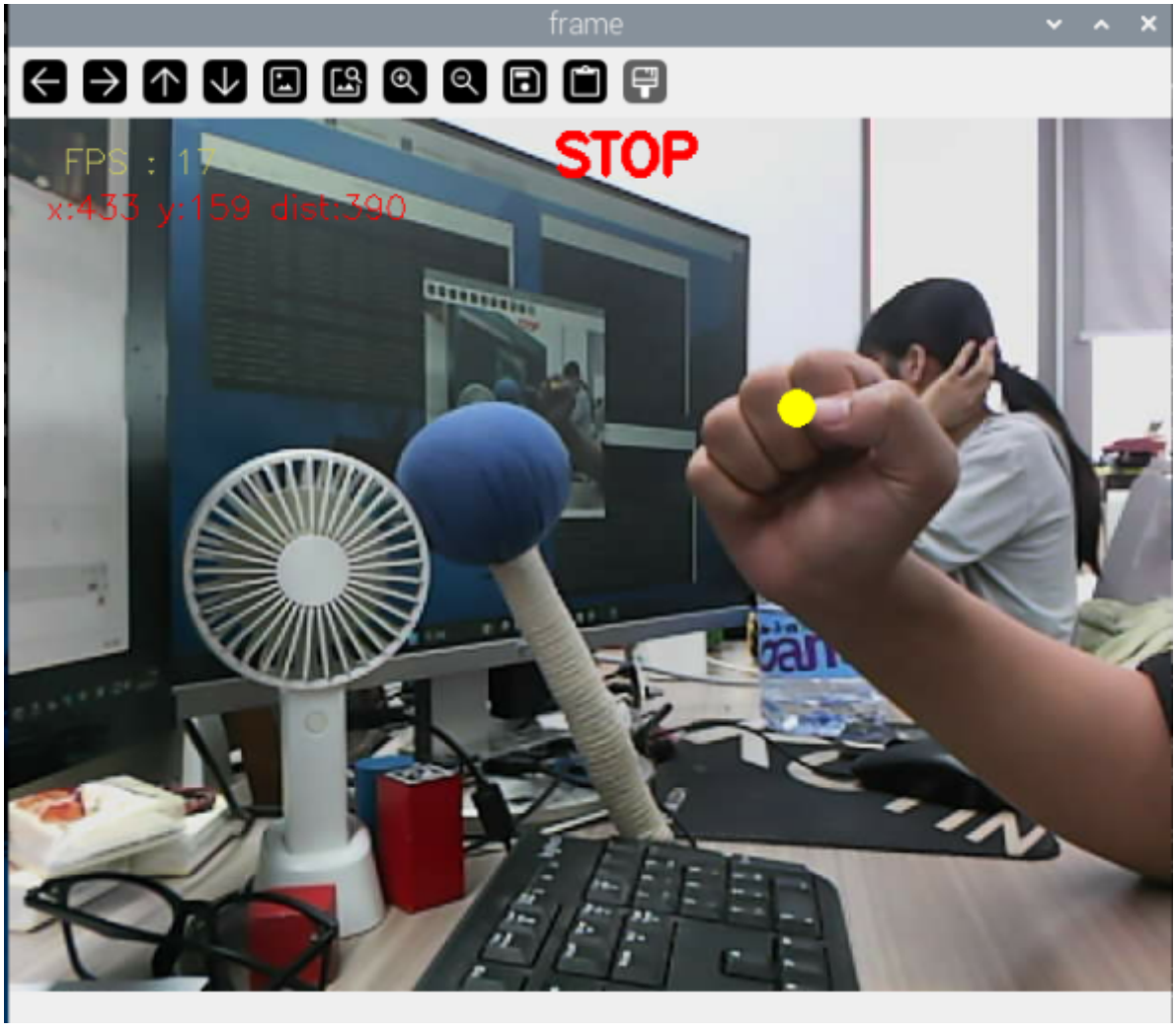
```
ros2 launch m3_bringup mediapipe_demo.launch.py demo:=gestureFollow
```

When a gesture is detected, it will automatically mark the gesture position and display the center coordinates, and the car will start tracking and following.

> [!WARNING]
>
> **Note:** The following effect is related to the palm depth distance information detected. If the effect is not good, you can increase the following distance or adjust the camera angle upward.

After the program starts, the following screen will appear:

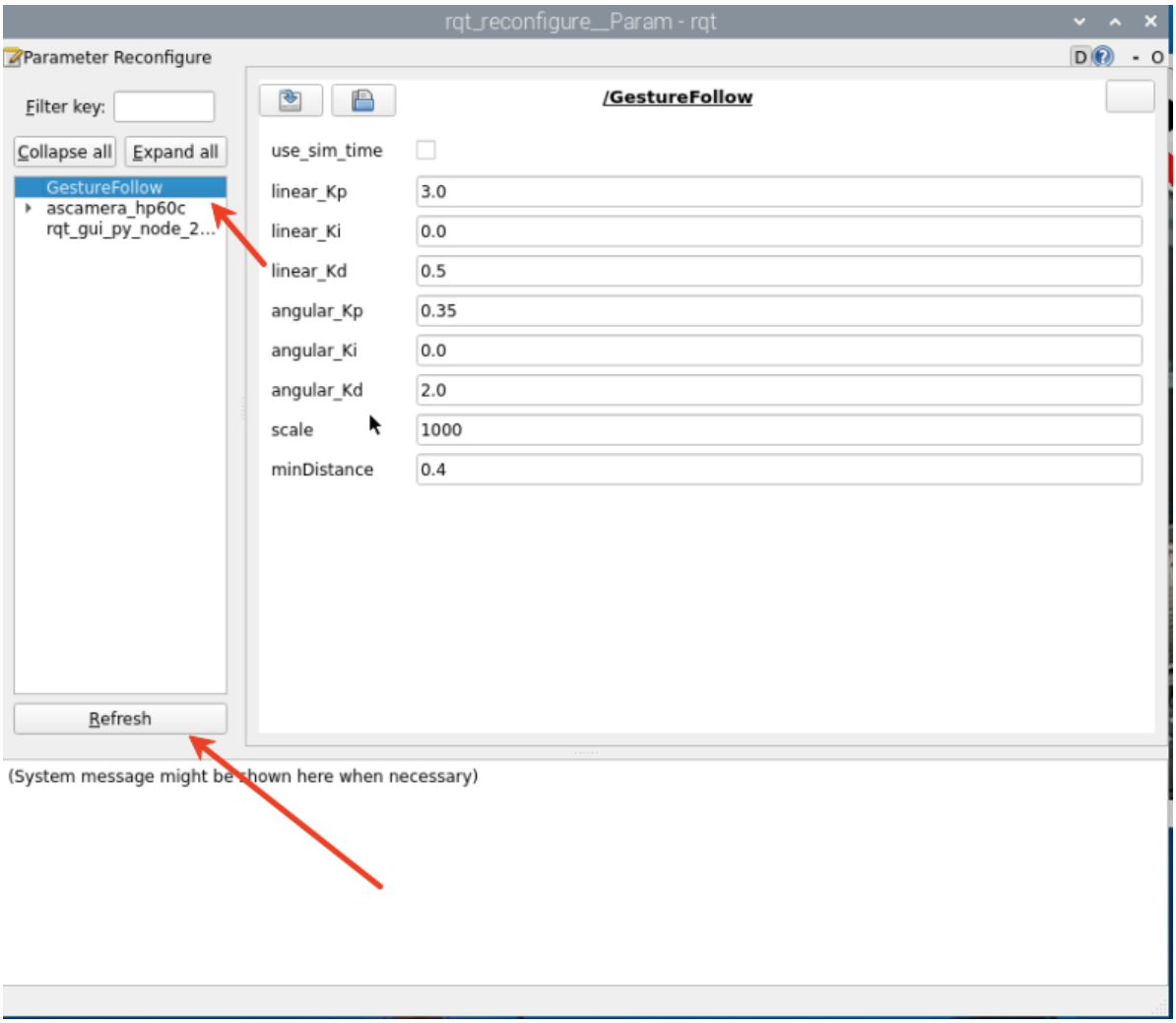

frame
RUN
FPS : 17
x:414 y:178 dist:494


frame
STOP
FPS : 17
x:433 y:159 dist:390

2.2 Dynamic Parameter Adjustment

Enter in terminal:

```
ros2 run rqt_reconfigure rqt_reconfigure
```



After modifying parameters, click on the blank area of the GUI to write the parameter values. Note that it will only take effect for the current startup. For permanent effect, the parameters need to be modified in the source code.

Parameter Analysis:

【linear_Kp】, 【linear_Ki】, 【linear_Kd】: Linear velocity PID control during car following.

【angular_Kp】, 【angular_Ki】, 【angular_Kd】: Angular velocity PID control during car following.

【minDistance】: Following distance, always maintain this distance.

【scale】: PID proportional scaling.

3. Source Code Analysis

Source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/mediapipe/12_gestureFol
low.py
```

This program mainly has the following functions:

- Initialize gesture detector
- Subscribe to depth topic to get images
- Detect gesture key points in images
- Calculate gesture center coordinates and publish
- Use PID algorithm to calculate servo angles, forward speed and publish control commands

Some core code is as follows:

```
#Initialize palm detection object
self.hand_detector = handDetector(detectorCon=0.8)

#距离计算 #Distance calculation
points = [
        (int(self.cy), int(self.cx)),         #Center point
        (int(self.cy +1), int(self.cx +1)),  #Bottom right offset point
        (int(self.cy - 1), int(self.cx - 1))   #Top left offset point
    ]
valid_depths = []
for y, x in points:
    depth_val = depth_image_info[y][x]
    if depth_val != 0:
        valid_depths.append(depth_val)
if valid_depths:
    dist = int(sum(valid_depths) / len(valid_depths))
else:
    dist = 0

#Determine forward movement or stop
if self.hand_detector.get_gesture() == "Zero":
    cv.putText(result_image, "STOP", (300, 30), cv.FONT_HERSHEY_SIMPLEX, 1, (0,
0, 255), 4)
    self.pub_cmdVel.publish(Twist())
elif dist > 0.2:
    cv.putText(result_image, "RUN", (300, 30), cv.FONT_HERSHEY_SIMPLEX, 1, (0,
0, 255), 4)
    self.execute(self.x,dist)

#Calculate the movement speed and turning angle using the PID algorithm based on
the x and y values
# Calculate the movement speed and turning angle using the PID algorithm based on
the x and y values
def execute(self, rgb_img, action):
    #PID calculation deviation
    linear_x = self.linear_pid.compute(dist, self.minDist)
    angular_z = self.angular_pid.compute(point_x,320)
    #The car is in the parking area and the speed is 0
    if abs(dist - self.minDist) < 30: linear_x = 0
    if abs(point_x - 320.0) < 30: angular_z = 0
    twist = Twist()
    ...
    #Publish the calculated speed information
    self.pub_cmdVel.publish(twist)
```